

Cahier des charges

Prototype d'admission d'un patient — Interopérabilité FHIR / HL7 v2

1. Contexte

Dans le cadre de mon évaluation d'interopérabilité en santé, je dois réaliser un prototype permettant de gérer **l'admission d'un patient**.

L'objectif n'est pas simplement de réaliser une application qui communique avec une API. Je dois montrer que je comprends les différentes dimensions de l'interopérabilité : la communication entre les systèmes, la structure des données, leur signification et leur transformation d'un standard à un autre.

Le prototype devra donc récupérer des informations depuis un **serveur FHIR réel**, puis les utiliser pour produire une représentation au format **HL7 v2**, qui est le format cible demandé pour cette évaluation.

Le serveur fourni pour le projet est :

HAPI FHIR R4

<https://hapi.fhir.org/baseR4>

La documentation Swagger du serveur est disponible à :

<https://hapi.fhir.org/baseR4/swagger-ui/>

Le serveur annonce la version **FHIR R4 – 4.0.1**. Il expose notamment les ressources **Patient**, **Encounter**, **Organization**, **Practitioner** et **Location**, qui sont pertinentes pour notre scénario.

2. Besoin métier

Le cas d'usage choisi est **l'admission d'un patient**.

L'idée est de partir des informations déjà disponibles dans le système FHIR et de les utiliser pour représenter une admission dans un autre format d'interopérabilité, ici HL7 v2.

Concrètement, l'utilisateur doit pouvoir :

1. rechercher un patient ;
2. sélectionner le patient concerné ;
3. récupérer ses informations depuis le serveur FHIR ;
4. récupérer les informations liées à son admission ou à son séjour ;
5. vérifier les données récupérées ;
6. préparer l'admission ;

7. transformer les informations FHIR nécessaires ;
8. générer une représentation HL7 v2 de l'admission ;
9. visualiser le mapping entre FHIR et HL7 v2.

L'application doit donc faire le lien entre :

FHIR → données du patient et de son séjour → mapping → HL7 v2

3. Objectifs du prototype

Le prototype doit permettre de démontrer les points suivants :

- savoir interroger un serveur FHIR réel ;
- comprendre et utiliser les ressources FHIR ;
- récupérer les données d'un patient ;
- identifier les données nécessaires à une admission ;
- comprendre la ressource `Encounter` ;
- faire le lien entre plusieurs ressources FHIR ;
- transformer les données FHIR vers HL7 v2 ;
- produire un message d'admission cohérent ;
- documenter le mapping ;
- détecter les informations qui ne peuvent pas être correctement transformées ;
- gérer les erreurs ;
- tester le comportement avec différentes données.

Le prototype doit privilégier la **compréhension et la démonstration de l'interopérabilité** plutôt qu'une architecture inutilement complexe.

4. Utilisateur et scénario d'utilisation

L'utilisateur représente une personne chargée de gérer l'admission administrative d'un patient.

Scénario principal

L'utilisateur arrive sur l'application.

Il recherche un patient à partir d'un critère disponible sur le serveur FHIR.

L'application interroge alors :

`https://hapi.fhir.org/baseR4`

et récupère les informations disponibles.

Après avoir sélectionné le patient, l'application récupère les informations nécessaires à son admission, notamment son `Encounter` lorsqu'il est disponible.

Les informations récupérées sont affichées afin que l'utilisateur puisse les vérifier.

L'application utilise ensuite les données nécessaires pour construire une représentation **HL7 v2 ADT^A01**, correspondant à une admission.

Enfin, l'utilisateur peut consulter :

- les données FHIR reçues ;
 - les données utilisées ;
 - le mapping FHIR → HL7 v2 ;
 - le message HL7 v2 généré ;
 - les éventuelles informations non mappées.
-

5. Périmètre

Dans le périmètre

Le prototype comprend :

- connexion au serveur HAPI FHIR ;
- vérification de la disponibilité du serveur ;
- recherche d'un patient ;
- récupération d'un **Patient** ;
- recherche/récupération d'un **Encounter** ;
- récupération éventuelle d'une **Organization** ;
- récupération éventuelle d'un **Practitioner** ;
- affichage des données ;
- préparation d'une admission ;
- génération d'une représentation HL7 v2 ;
- mapping FHIR → HL7 v2 ;
- gestion des erreurs ;
- traçabilité minimale ;
- tests.

Hors périmètre

Le prototype ne cherche pas à remplacer un véritable logiciel hospitalier.

Ne sont donc pas prévus :

- gestion complète du dossier médical ;
- prescription ;
- facturation ;
- gestion complète des rendez-vous ;
- gestion complète des médicaments ;
- gestion des résultats biologiques ;
- gestion d'un parcours de soins complet ;
- mise en production hospitalière.

6. Ressources FHIR utilisées

6.1 Patient

La ressource `Patient` représente la personne admise.

C'est la ressource principale pour récupérer les informations d'identité.

Les données susceptibles d'être utilisées sont :

- `id` ;
- `identifier` ;
- `name` ;
- `birthDate` ;
- `gender` .

Selon les données réellement présentes dans le serveur, d'autres informations pourront être affichées, mais elles ne doivent pas être utilisées dans le mapping si elles ne sont pas nécessaires.

6.2 Encounter

`Encounter` est la ressource centrale pour représenter l'interaction entre le patient et le système de soins.

Elle est donc particulièrement importante pour notre cas d'usage d'admission.

Les informations susceptibles d'être utilisées sont notamment :

- `id` ;
- `status` ;
- `class` ;
- `type` ;
- `subject` ;
- `period` ;
- `participant` ;
- `serviceProvider` .

La relation principale est :

Encounter.subject
↓
Patient

Cette relation permet de relier le séjour/admission au patient.

6.3 Organization

`Organization` pourra être utilisée lorsque l'Encounter fait référence à une organisation.

Elle pourra notamment représenter :

- l'établissement ;
- l'organisation responsable ;
- éventuellement une structure de soins.

Elle ne sera utilisée que si elle apporte réellement une information utile au scénario.

6.4 Practitioner

`Practitioner` pourra être utilisé si l'Encounter contient un professionnel participant à l'admission ou au séjour.

Il ne sera pas obligatoire si le scénario d'admission ne nécessite pas cette information.

6.5 Location

La ressource `Location` est disponible sur le serveur.

Elle pourra être utilisée si l'admission nécessite de représenter une localisation précise du patient : unité, chambre, service, etc.

Elle ne sera pas ajoutée artificiellement si les données disponibles ne l'utilisent pas.

Le serveur expose bien cette ressource.

7. Version FHIR

Le serveur fourni utilise :

FHIR R4 — version 4.0.1.

Le prototype devra donc être développé en respectant la spécification FHIR R4.

La première étape du développement devra néanmoins consister à consulter les capacités réelles du serveur afin de confirmer :

- les ressources utilisables ;
- les opérations disponibles ;
- les paramètres de recherche ;
- les profils ;

- les éventuelles contraintes particulières.

Le serveur fournit également un `CapabilityStatement`, ce qui permettra de vérifier ses capacités réelles.

8. Interactions avec le serveur FHIR

8.1 Vérifier le serveur

L'application devra commencer par vérifier que le serveur est accessible.

L'opération de référence sera :

```
GET https://hapi.fhir.org/baseR4/metadata
```

L'objectif est notamment de vérifier les capacités et la version annoncée du serveur.

8.2 Rechercher un patient

L'application devra permettre de rechercher un patient.

L'endpoint général sera :

```
GET https://hapi.fhir.org/baseR4/Patient
```

Les paramètres de recherche utilisés devront être ceux effectivement supportés par le serveur.

Par exemple, selon les capacités disponibles :

```
Patient?name=...  
Patient?identifier=...
```

Le prototype ne devra pas supposer qu'un paramètre est disponible sans le vérifier.

8.3 Récupérer un patient

Une fois le patient sélectionné :

```
GET https://hapi.fhir.org/baseR4/Patient/{id}
```

L'application devra récupérer la ressource complète et utiliser cette réponse comme source des données affichées.

8.4 Rechercher un Encounter

Pour rechercher les admissions/séjours associés au patient :

```
GET https://hapi.fhir.org/baseR4/Encounter?subject=Patient/{id}
```

Le résultat devra être analysé afin de sélectionner l'Encounter pertinent pour le scénario.

8.5 Récupérer une Organization

Si l'Encounter contient une référence vers une organisation :

```
GET https://hapi.fhir.org/baseR4/Organization/{id}
```

8.6 Récupérer un Practitioner

Si nécessaire :

```
GET https://hapi.fhir.org/baseR4/Practitioner/{id}
```

9. Gestion des réponses FHIR

L'application devra vérifier que les réponses reçues sont bien celles attendues.

Par exemple :

```
Patient
Encounter
Organization
Practitioner
```

Une réponse inattendue ne devra pas être traitée comme si elle était valide.

L'application devra également gérer :

- résultat vide ;
- ressource absente ;
- champ absent ;
- référence incorrecte ;
- erreur HTTP ;
- serveur indisponible ;
- réponse mal formée.

10. Format HL7 v2

Le format cible du projet est **HL7 v2**.

Pour notre scénario d'admission, le message retenu sera :

ADT^A01

Il correspond au scénario d'admission d'un patient.

La structure minimale étudiée sera notamment :

MSH
EVN
PID
PV1

D'autres segments ne seront ajoutés que s'ils sont réellement nécessaires.

11. Rôle des segments HL7 v2

MSH

MSH contient les informations générales du message :

- système émetteur ;
- système destinataire ;
- date/heure ;
- type de message ;
- identifiant du message ;
- version HL7 v2.

Les valeurs exactes devront être définies en fonction du contexte du prototype.

EVN

EVN décrit l'événement.

Dans notre scénario :

A01

Il permettra notamment de représenter l'événement d'admission et les informations temporelles associées.

PID

PID contient les informations d'identification du patient.

Il sera principalement alimenté à partir de :

FHIR Patient

PV1

PV1 contient les informations liées au séjour/admission.

Il sera principalement alimenté à partir de :

FHIR Encounter

et éventuellement de ressources liées.

12. Mapping FHIR → HL7 v2

Le mapping devra être documenté avant l'implémentation.

Ressource FHIR	Champ	Signification	HL7 v2	Transformation	Risque
Patient	identifier	Identifiant patient	PID-3	Conversion vers identifiant HL7	Plusieurs identifiants possibles

Ressource FHIR	Champ	Signification	HL7 v2	Transformation	Risque
Patient	name.family	Nom	PID-5	Copie	Structure différente
Patient	name.given	Prénom	PID-5	Copie	Plusieurs prénoms possibles
Patient	birthDate	Date de naissance	PID-7	Format adapté	Faible
Patient	gender	Sexe administratif	PID-8	Mapping de codes	Terminologie à vérifier
Encounter	id	Identifiant du séjour	PV1	Transformation selon besoin	À vérifier
Encounter	status	Statut du séjour	PV1 / autre champ	Mapping	Correspondance à vérifier
Encounter	class	Classe de prise en charge	PV1-2	Mapping de codes	Terminologie différente
Encounter	period.start	Début du séjour	EVN/ PV1	Conversion date/heure	À vérifier
Encounter	serviceProvider	Organisation responsable	PV1 / autre champ	Référence → identifiant	À vérifier
Encounter	participant	Professionnel	PV1	Référence → identifiant	Peut être absent
Organization	identifier	Identifiant établissement	MSH/ PV1 selon contexte	Transformation	À vérifier
Organization	name	Nom établissement	MSH/ PV1 selon contexte	Copie	Sémantique à vérifier

Ce tableau est une **base de travail**.

Il devra être vérifié avec la spécification HL7 v2 retenue et les données réellement présentes dans les ressources du serveur.

13. Règle de mapping

Il est interdit de créer un mapping uniquement parce que deux champs semblent avoir une signification similaire.

Chaque correspondance doit être classée comme :

- correspondance exacte ;
- transformation nécessaire ;
- correspondance approximative ;
- correspondance non déterminée.

Si une correspondance fiable n'est pas possible :

« **Correspondance non déterminée** »

doit être indiqué.

Aucune donnée ne doit être inventée pour compléter le message.

14. Interopérabilité sémantique

Le projet devra distinguer trois niveaux.

Syntaxe

Est-ce que les données sont correctement représentées techniquement ?

Exemples :

- JSON FHIR ;
- segments HL7 v2 ;
- séparateurs HL7.

Structure

Est-ce que les informations sont organisées correctement ?

Exemple :

```
Patient
↓
Encounter
↓
Organization
```

vers :

MSH
EVN
PID
PV1

Sémantique

Est-ce que les informations gardent le même sens après transformation ?

C'est le point le plus important.

Un message peut être syntaxiquement correct tout en étant sémantiquement faux.

15. Terminologies

Une attention particulière devra être portée aux codes.

Les éléments suivants devront être vérifiés :

- sexe ;
- classe d'Encounter ;
- établissement ;
- service ;
- rôle du professionnel ;
- identifiants.

Pour chaque code, il faudra connaître :

1. sa valeur ;
2. son système de codification ;
3. sa signification ;
4. sa correspondance éventuelle dans HL7 v2.

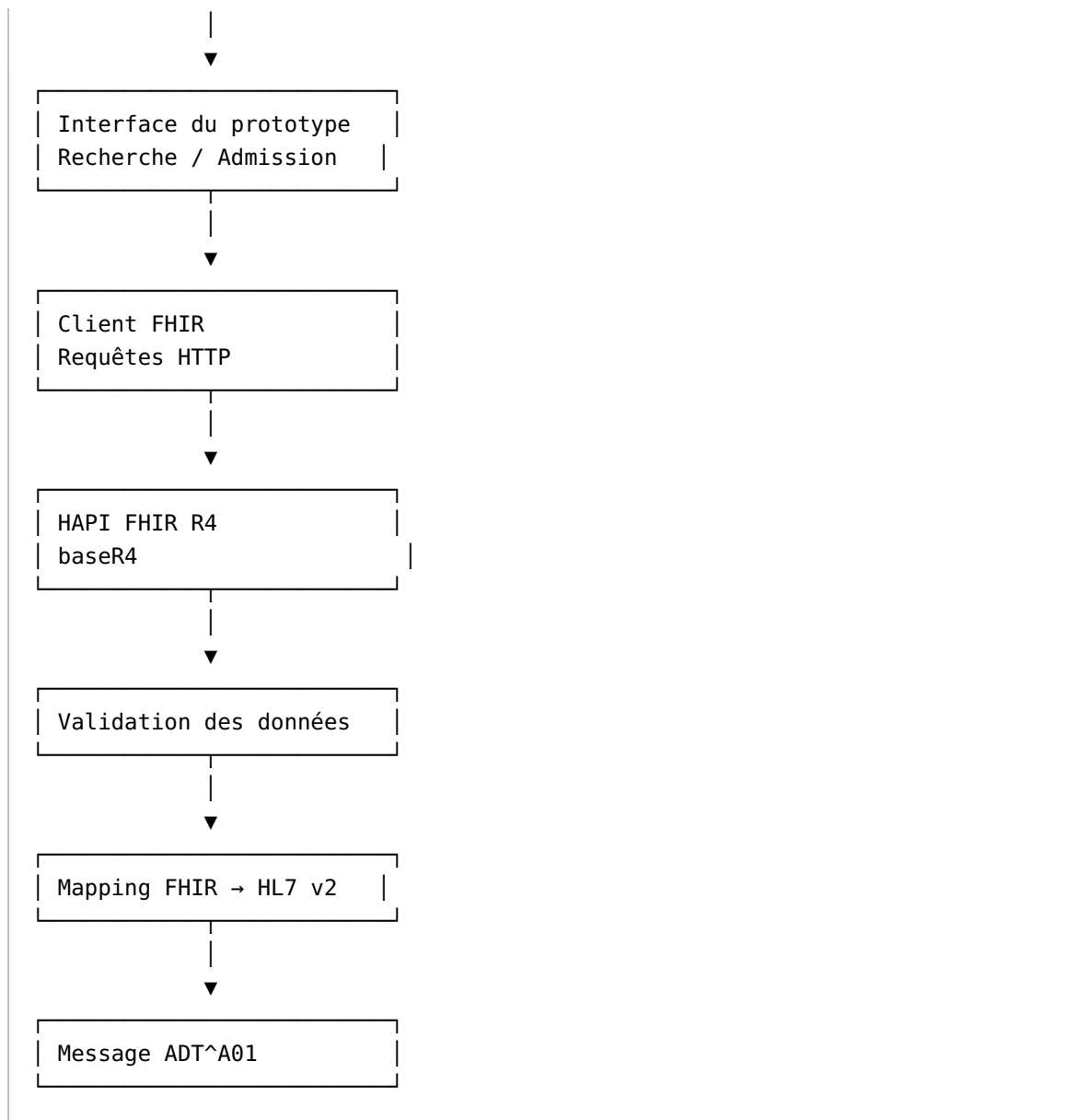
Une simple traduction textuelle ne sera pas considérée comme suffisante.

16. Architecture proposée

L'architecture du prototype pourra rester simple :

```
graph TD; U[Utilisateur];
```

Utilisateur



17. Analyse selon le ReEIF

17.1 Juridique

Le prototype manipule des données de santé.

Il faudra donc prendre en compte :

- confidentialité ;
- protection des données ;
- minimisation ;
- limitation de la conservation ;
- traçabilité.

Pour une évaluation, il faudra vérifier les règles applicables au serveur fourni.

17.2 Politique / gouvernance

Il faudra déterminer :

- qui est autorisé à accéder aux données ;
 - quelles données peuvent être utilisées ;
 - quelles règles de partage sont applicables ;
 - quelles terminologies sont imposées.
-

17.3 Processus de soin

Le processus étudié est :

```
Recherche patient
  ↓
Identification
  ↓
Sélection
  ↓
Récupération Encounter
  ↓
Admission
  ↓
Production ADT^A01
  ↓
Vérification
```

17.4 Information

Cette couche concerne :

- Patient ;
 - Encounter ;
 - Organization ;
 - Practitioner ;
 - Location ;
 - identifiants ;
 - dates ;
 - codes.
-

17.5 Applications

Cette couche comprend :

- interface utilisateur ;
 - client FHIR ;
 - validation ;
 - mapping ;
 - génération HL7 v2 ;
 - gestion des erreurs ;
 - traçabilité.
-

17.6 Infrastructure

Elle comprend :

- poste de développement ;
 - réseau ;
 - HTTP/HTTPS ;
 - serveur HAPI FHIR ;
 - environnement d'exécution.
-

18. Contraintes techniques

Le prototype devra :

- utiliser le serveur HAPI FHIR réel ;
 - utiliser FHIR R4 ;
 - ne pas remplacer les données par des données fictives ;
 - respecter les ressources réellement disponibles ;
 - gérer les erreurs HTTP ;
 - gérer les réponses FHIR inattendues ;
 - afficher les données récupérées ;
 - produire un ADT^A01 ;
 - permettre de vérifier le mapping ;
 - ne pas coder en dur un patient spécifique ;
 - fonctionner avec plusieurs patients ;
 - conserver une trace minimale des échanges.
-

19. Gestion des erreurs

Les erreurs devront être compréhensibles pour l'utilisateur.

Exemples :

Serveur inaccessible

Impossible de contacter le serveur FHIR.

Patient inexistant

Aucun patient ne correspond à votre recherche.

Ressource absente

Les informations d'admission nécessaires ne sont pas disponibles.

Donnée manquante

Une information nécessaire à la génération du message HL7 v2 est absente.

Mapping impossible

Correspondance non déterminée entre cette donnée FHIR et le format HL7 v2.

Erreur serveur

Le serveur FHIR a retourné une erreur. Veuillez réessayer.

20. Traçabilité

Le prototype devra conserver une trace minimale des échanges.

Pour chaque opération importante :

Date / heure
Opération
Ressource
Endpoint
Code HTTP
Résultat
Erreur éventuelle

Exemple :

GET /Patient/123
HTTP 200
Patient récupéré

La traçabilité devra cependant éviter de stocker inutilement des données personnelles.

21. Critères d'acceptation

AC01 — Connexion

Étant donné que le serveur HAPI FHIR est accessible,

Lorsque l'application démarre,

Alors elle doit pouvoir vérifier que le serveur est disponible.

AC02 — Version

Étant donné que le serveur annonce FHIR R4,

Alors l'application doit utiliser des ressources conformes à FHIR R4.

AC03 — Recherche

L'utilisateur doit pouvoir rechercher un patient existant.

AC04 — Patient inexistant

Une recherche sans résultat doit être signalée clairement.

AC05 — Données réelles

Les informations affichées doivent provenir du serveur FHIR.

AC06 — Encounter

L'application doit pouvoir récupérer ou rechercher l'Encounter associé au patient lorsque celui-ci existe.

AC07 — Admission

Lorsqu'un scénario d'admission valide est constitué, l'application doit produire une représentation

ADT^A01.

AC08 — Mapping

Chaque donnée utilisée dans le message HL7 v2 doit pouvoir être reliée à son origine FHIR.

AC09 — Donnée manquante

Une donnée absente ne doit jamais être inventée.

AC10 — Erreur

Une erreur HTTP ou FHIR doit être détectée et présentée correctement.

AC11 — Terminologie

Les codes utilisés doivent être vérifiés par rapport à leur système de codification.

AC12 — Traçabilité

Les échanges importants doivent être traçables.

22. Plan de tests

ID	Objectif	Entrée	Action	Résultat attendu
T01	Vérifier serveur	URL HAPI	GET metadata	Serveur accessible
T02	Vérifier version	Metadata	Lecture CapabilityStatement	FHIR 4.0.1
T03	Rechercher patient	Critère valide	GET Patient	Résultat
T04	Patient inexistant	Critère invalide	GET Patient	Aucun résultat
T05	Récupérer Patient	ID valide	GET	Patient valide
T06	Rechercher Encounter	Patient ID	GET Encounter	Encounter retourné
T07	Patient incomplet	Ressource incomplète	Traitement	Donnée manquante détectée
T08	Endpoint incorrect	URL invalide	GET	Erreur détectée

ID	Objectif	Entrée	Action	Résultat attendu
T09	Serveur indisponible	URL inaccessible	Appel	Message d'erreur
T10	Générer ADT	Patient + Encounter	Mapping	ADT^A01
T11	Vérifier PID	Patient	Mapping	PID cohérent
T12	Vérifier PV1	Encounter	Mapping	PV1 cohérent
T13	Vérifier EVN	Date/événement	Mapping	EVN cohérent
T14	Vérifier terminologie	Code FHIR	Transformation	Code vérifié
T15	Vérifier traçabilité	Échange	Consultation trace	Trace disponible
T16	Deuxième patient	Autre patient	Scénario complet	Pas de donnée codée en dur

23. Vérification des propositions de l'IA

L'IA sera utilisée comme assistant de développement et non comme source de vérité.

Avant d'accepter une proposition, il faudra vérifier :

- la version FHIR ;
- la ressource utilisée ;
- le champ utilisé ;
- l'endpoint ;
- les paramètres ;
- la structure HL7 v2 ;
- le segment et le champ HL7 ;
- les terminologies ;
- les règles de mapping.

Les informations issues de l'IA devront être comparées aux spécifications et aux réponses réelles du serveur.

24. Stratégie de développement

Le développement suivra la boucle :

```

CONCEVOIR
↓
DEMANDER À L'IA
↓

```

```
graph TD; A[VÉRIFIER] --> B[TESTER SUR HAPI FHIR]; B --> C[IDENTIFIER L'ERREUR]; C --> D[CORRIGER]; D --> E[RETESTER]; E --> F[VALIDER];
```

Chaque étape doit être vérifiable.

Une fonctionnalité ne sera pas considérée comme terminée simplement parce qu'elle fonctionne techniquement.

Elle devra également être cohérente avec :

- FHIR ;
- HL7 v2 ;
- les données réelles ;
- les terminologies ;
- le scénario métier.

25. Limites et pertes d'information

Plusieurs difficultés peuvent apparaître.

Identifiants

Un patient FHIR peut avoir plusieurs identifiants.

Il faudra déterminer lequel correspond à l'identifiant attendu dans HL7 v2.

Nom

FHIR permet de représenter plusieurs composants du nom.

Une transformation peut être nécessaire vers la structure HL7 v2.

Sexe

Les valeurs FHIR et HL7 v2 peuvent utiliser des systèmes de codification différents.

Une table de correspondance devra être vérifiée.

Encounter

FHIR permet de représenter beaucoup d'informations sur un séjour.

HL7 v2 ne possède pas nécessairement un équivalent direct pour chaque élément.

Certaines informations pourront donc être perdues.

Organisation

Une `Organization` FHIR peut contenir plus d'informations que ce qui est nécessaire dans le message HL7.

Il faudra sélectionner uniquement les informations utiles.

26. Points restant à vérifier sur les données réelles

Même si le serveur est maintenant identifié comme **FHIR R4**, il faudra encore vérifier les données concrètes utilisées pour notre scénario.

Avant de coder, nous devons notamment vérifier :

- quels patients sont facilement recherchables ;
 - quels champs sont présents dans leurs ressources `Patient` ;
 - quels patients possèdent un `Encounter` ;
 - quelles informations sont présentes dans les `Encounter` ;
 - si les `Encounter` contiennent une `Organization` ;
 - si des `Practitioner` sont associés ;
 - si une `Location` est disponible ;
 - quelles terminologies sont réellement utilisées ;
 - quelles valeurs peuvent être utilisées pour construire le message ADT^A01.
-

27. Validation finale du cahier des charges

Avant de commencer le développement, les points suivants devront être validés :

FHIR

- [x] Serveur identifié
- [x] HAPI FHIR
- [x] FHIR R4
- [x] Version 4.0.1
- [x] Patient disponible
- [x] Encounter disponible

- ☒ Organization disponible
- ☒ Practitioner disponible
- ☒ Location disponible
- ☐ Données exactes du patient à sélectionner
- ☐ Structure exacte de l'Encounter choisi
- ☐ Terminologies réellement présentes

HL7 v2

- ☒ Format cible identifié
- ☒ ADT^A01 retenu pour le scénario d'admission
- ☒ MSH identifié
- ☒ EVN identifié
- ☒ PID identifié
- ☒ PV1 identifié
- ☐ Version HL7 v2 exacte à utiliser
- ☐ Convention exacte des champs à confirmer

Mapping

- ☒ Principe défini
- ☒ Patient → PID
- ☒ Encounter → PV1/EVN
- ☐ Mapping final à vérifier avec les données réelles
- ☐ Terminologies à vérifier
- ☐ Pertes d'information à documenter

28. Conclusion

Le prototype doit finalement démontrer cette chaîne :

```

Serveur HAPI FHIR R4
    ↓
Recherche du patient
    ↓
Patient
    ↓
Encounter
    ↓
Validation des données
    ↓
Mapping FHIR → HL7 v2
    ↓
ADT^A01
    ↓
Vérification du résultat
    ↓
Validation de l'interopérabilité
  
```

L'objectif n'est donc pas seulement de montrer :

« J'arrive à appeler une API FHIR. »

Il faut pouvoir montrer :

« Je sais récupérer une information réelle, comprendre sa signification, la relier au processus d'admission, la transformer correctement vers HL7 v2 et vérifier que le sens de l'information est conservé. »

Le développement ne commencera qu'après validation du mapping et des données réellement disponibles sur le serveur.

La démarche retenue sera :

Concevoir → Vérifier → Implémenter → Tester → Corriger → Retester → Valider.